Page No.
Date

Pillars of OOPS | Access Modifier | Packages

* Access Modifier

In Java there Four Access Modifiers
- ↳ Public
- ↳ Default
- ↳ Protected
- ↳ Private

Access modifiers are
- ↳ who have access to
  - — variables
  - — Methods
  - — constructor
  - — class

1. ~~Private~~ Lets take example of Bank which contains data such as balance, IFSC code etc. To restrict access to balance we can use Access Modifier.

1. Private
   - ↳ specified using Private keyword.
   - ↳ The methods or data members declared as Private are accessible only within class in which they are declared.

eg:

```
class BankAccount {
        Private double balance;
        Priva void withdraw() {
                balance--;
        }
}
```

* Package group similar classes / Interface together.

2. Default
↳ If no access modifier is specified, the member has default access and can only be accessed within same package.
↳ Anyone can access belonging to same package.

eg:

```
class car {
    String model;    // default access.
}

public class main {
    public static void main (String[] args)
    {
        car c = new car();
        c.model = "Tesla";   // accessible within
        S.o.pln (c.model);   same package.
    }
```

3. Protected.
↳ specified using protected keyword.
↳ accessible within same package or subclass in diff package. (child class)

4. Public
↳ specified using public keyword
↳ public members are accessible from everywhere. Any one ~~class~~ can access.

✓ Private | default | protected | ✓ Public

Page No.
Date

Packages in Java

- A Package in Java is used to group related classes, interfaces and sub-packages.
- It used to avoid name conflicts, group related code and improve code maintainability and access control.
- Java has two types of JE Packages

Built-in Package	User-defined Package
(like java.util, java.io, java.lang)	(created by the programmer)

- syntax to declare package
  Package package-name;

- syntax to import package
  1. import package-name.class-name;
     ↳ It used to import only specific class from a package.
  2. import package_name.*;
     ↳ It is used to import all classes from package

- Four Pillars of OOPS
  ↳ Encapsulation
  ↳ Inheritance
  ↳ Polymorphism
  ↳ Abstraction

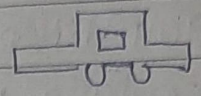
Page No.
Date

1. Encapsulation

- Mechanism of binding data (variables) and actions (methods) into a single unit called class
- Technically, every class is an example of Encapsulation.

Real world Example:

A capsule in which main medicine is encapsulated.



A car in which engine wheels & other parts are encapsulated.

Program

```
class Animal{
    int no. of legs ; } data
    String type ;
    void eat () { } method.
}
}
```

2. Rules or how to achieve encapsulation in Java

1. Private variable
   - Declare variable as Private so that they cannot be accessed directly from outside the class.
2. Public Getter & setter Method
   Provide Public getter & setter Method to access & Modify Private variable.

eg:

```
class Account {
    Private double balance;
    Public double get balance() {
        return balance;
    }

    Public void deposit (double amount) {
        if (amount > 0)
        balance = balance + amount;
    }
```

2. Inheritance

- Inheritance means acquiring the properties and behaviour of a parent class in a child class.
- Allows a subclass (child class) to inherit fields & method from superclass (parent class). Promoting code reusability & Method overriding (
- Represents → Is-a relationship (parent child relationship

eg: A car is-A vehicle

A dog is-A Animal

- How to acheive Inheritance in Java?

- By using the extend keyword for class inhe
- By using the implement keyword for interface inheritance

In realworld
how two objects are related — Student (is-a) → engineering st.

vertical (is a) → car

Inheritance
└ code Reusability
└ support Polymorphism

code:

```
class student {        — Parent class
    String name;
    void Attendlec()
    { s.o.pln("lec Attended");
    }
}
class Engstud Extends student {        — subclass
        void Attendlab() {
            s.o.pln("Attended lab");
        }
}
class demo {
    p.s.v.m (String args[]) {
        Engstud es = new engstud();
            es.Attendlab();
            es.Attendlec();
}   }
```

* Types of Inheritance
↳ single Inheritance
↳ Multilevel Inheritance
↳ Hierarchical Inheritance
↳ Multiple Inheritance
↳ hybrid Inheritance.